Report

My code creates a Metro Trip Planner that uses Dijkstra's algorithm to efficiently determine the optimal route between two metro stops. The code consists of several classes, including Main, JourneyPlanner, Graph, Stop, and Edge. The master class serves as the entry point that coordinates the trip planning process.

The JourneyPlanner class handles the graphical representation of the subway system and contains methods for reading subway data from a CSV file and finding the best journey in terms of minimum time. The Graph class maintains lists of Stop and Edge objects that represent individual subway stops and the weighted connections between them. The Stop class contains the characteristics of a subway stop, while the Edge class specifies predominant connections.

Despite a solid foundation, the code could benefit from improvements such as improved error handling in the findBestJourneyMinTime method, clearer variable naming for better readability, and the adoption of efficient stop search using a Map in the JourneyPlanner class. Other considerations include comprehensive error handling for edge cases such as empty or malformed CSV files, and experimentation with resources for proper resource management during file reading.

In summary, the code effectively implements Metro Trip Planner but offers opportunities for improvement in error handling, variable naming, and resource management. Addressing these areas will contribute to the clarity, reliability and maintainability of the code.